Quanta — Getting Started

Prerequisites

Make sure you have the following installed:

- Python 3.10+
- Git
- Docker & Docker Compose *(only required if you use PostgreSQL, Neo4j, or Redis)*

Step 1 — Clone the repository

```
git clone https://github.com/<username>/Quanta.git
cd Quanta
```

Step 2 — Create a virtual environment

```
python -m venv .venv
```

Activate it:

```
# macOS / Linux
source .venv/bin/activate

# Windows (PowerShell)
.venv\Scripts\Activate.ps1

# Windows (CMD)
.venv\Scripts\activate.bat
```

Verify you are using the correct Python:

```
which python   # macOS/Linux — should point into .venv
python --version
```

> `.venv/` is already in `.gitignore` and will not be committed.

Step 3 — Install the library

```
# Core — vector search + DuckDB docstore (no external services needed)
pip install -e .

# With LlamaIndex integration
pip install -e ".[llama-index]"

# With Neo4j graph expansion
pip install -e ".[neo4j]"

# With Redis embedding cache
pip install -e ".[cache]"

# With Tantivy BM25 full-text search
pip install -e ".[bm25]"

# Everything
pip install -e ".[neo4j,llama-index,cache,bm25]"

# Development (tests, linting, type-checking)
pip install -e ".[neo4j,llama-index,cache,bm25,dev]"
```

Step 4 — Verify the installation

```
python -c "import quanta; print('OK')"
```

Step 5 — Run the tests

```
pytest tests/ -v
```

All tests run without external services (they use DuckDB and mocks).

Step 6 — First use (zero services)

No `.env` file or Docker required. The DuckDB docstore is included in the core install.

```
import asyncio
import numpy as np
from quanta import QuantaIndex, DuckDBDocStore, MultiRetriever, NullGraph,
QuantaSettings
import os

async def main():
```

```
    # POSTGRES_USER and POSTGRES_PASSWORD are always validated — set placeholders
    # when using DuckDB so no PostgreSQL connection is attempted.
    os.environ.setdefault("POSTGRES_USER", "_unused_")
    os.environ.setdefault("POSTGRES_PASSWORD", "_unused_")

    settings = QuantaSettings(DOCSTORE_BACKEND="duckdb",
DUCKDB_PATH="./demo.duckdb")

    docstore = DuckDBDocStore(settings)
    await docstore.init()

    idx = QuantaIndex(name="text", dim=768)
    retriever = MultiRetriever(
        indexes={"text": idx},
        docstore=docstore,
        graph=NullGraph(),
    )

    # Add a document and its chunk
    await docstore.add_document("doc-1", "Hello world.", "text")
    await docstore.add_chunk("chunk-1", "doc-1", "Hello world.", chunk_index=0)
    idx.add(np.random.rand(1, 768).astype(np.float32), ["chunk-1"])

    # Search
    query = np.random.rand(768).astype(np.float32)
    results = await retriever.search(query_vectors={"text": query}, k=3)
    for r in results:
        print(f"[{r.score:.4f}] [{r.source}] {r.content}")

    await docstore.close()

asyncio.run(main())
```

Step 7 — With PostgreSQL (optional)

Copy `.env.example` to `.env` and fill in your values:

```
cp .env.example .env
```

Minimum required variables:

```
# ── PostgreSQL ──────────────────────────────────────────────────────
POSTGRES_HOST=localhost
POSTGRES_PORT=5432
POSTGRES_DB=quanta
POSTGRES_USER=quantauser
POSTGRES_PASSWORD=changeme
```

```
# ── Neo4j (optional — leave blank if you are not using graph retrieval) ─
NEO4J_URI=bolt://localhost:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=changeme_neo4j
NEO4J_DATABASE=neo4j

# ── Redis (optional — leave blank to disable embedding cache) ──────────
REDIS_HOST=localhost

# ── Index defaults ──────────────────────────────────────────────────────
DEFAULT_BIT_WIDTH=4
DEFAULT_TOP_K=10
```

> Never commit `.env` to git. It is already listed in `.gitignore`.

Start the services you need:

```
# PostgreSQL only
docker compose up -d postgres

# PostgreSQL + Neo4j (graph expansion)
docker compose --profile graph up -d

# PostgreSQL + Redis (embedding cache)
docker compose --profile cache up -d

# Everything
docker compose --profile graph --profile cache up -d

# Verify all services are healthy
docker compose ps
```

Step 8 — With PostgreSQL (full pipeline)

```
import asyncio
import numpy as np
from quanta import QuantaIndex, DocStore, MultiRetriever, NullGraph,
QuantaSettings

async def main():
    settings = QuantaSettings()          # reads from .env
    docstore = DocStore(settings)        # DocStore is an alias for
PostgresDocStore
    await docstore.init()                # creates tables on first run

    idx = QuantaIndex(name="articles", dim=768)
    retriever = MultiRetriever(
        indexes={"articles": idx},
```

```
        docstore=docstore,
        graph=NullGraph(),
        dense_weight=1.0,
    )

    # Index a document
    await docstore.add_document(
        "art-1", "Full article text here.", "article",
        metadata={"year": 2024, "topic": "AI"},
    )
    await docstore.add_chunk(
        id="art-1-c0", document_id="art-1",
        content="Full article text here.",
        chunk_index=0, metadata={"year": 2024, "topic": "AI"},
    )
    idx.add(np.random.rand(1, 768).astype(np.float32), ["art-1-c0"])

    # Filtered search — only chunks with topic=AI
    results = await retriever.search(
        query_vectors={"articles": np.random.rand(768).astype(np.float32)},
        k=5,
        filters={"topic": "AI"},
    )
    for r in results:
        print(f"[{r.score:.4f}] {r.id}  doc={r.document_id}")

    await docstore.close()

asyncio.run(main())
```

Step 9 — Neo4j Browser (optional)

If you started Neo4j, open in your browser:

```
http://localhost:7474
```

Log in with `neo4j` / the password you set in `.env`.

Step 10 — LlamaIndex integration (optional)

```
pip install -e ".[llama-index]"
```

```
import asyncio
import numpy as np
from quanta import QuantaIndex, DocStore, MultiRetriever, NullGraph,
```

```
QuantaSettings
from quanta.integrations.llama_index import QuantaVectorStore

from llama_index.core import VectorStoreIndex, StorageContext
from llama_index.core.schema import TextNode

DIM = 768

async def main():
    settings = QuantaSettings()
    docstore = DocStore(settings)
    await docstore.init()

    idx = QuantaIndex(name="llamaidx", dim=DIM)
    retriever = MultiRetriever(
        indexes={"llamaidx": idx},
        docstore=docstore,
        graph=NullGraph(),
    )

    store = QuantaVectorStore(retriever=retriever, index_name="llamaidx",
embed_dim=DIM)
    storage_ctx = StorageContext.from_defaults(vector_store=store)
    li_index = VectorStoreIndex(nodes=[], storage_context=storage_ctx)

    # Add a node (embedding must be pre-computed)
    node = TextNode(
        node_id="doc-1",
        text="Quanta enables hybrid search with graph expansion.",
        embedding=np.random.rand(DIM).tolist(),
        metadata={"source": "manual"},
    )
    await store.async_add([node])

    # Query
    query_engine = li_index.as_query_engine()
    # (Pass a real query embedding to query_engine or use store.aquery directly)

    await docstore.close()

asyncio.run(main())
```

Useful Docker commands

```
# Stop all services
docker compose down

# Stop and delete volumes (WARNING: all data is lost)
docker compose down -v
```

```
# Follow PostgreSQL logs
docker compose logs -f postgres

# Follow Neo4j logs
docker compose logs -f neo4j

# Restart PostgreSQL only
docker compose restart postgres
```

Linting & type checking

```
# Ruff
ruff check quanta/

# Mypy
mypy quanta/

# Both together
ruff check quanta/ && mypy quanta/
```

Common problems

`Connection refused` **on PostgreSQL** → Check the service is running: `docker compose ps` → Wait 5–10 seconds after `docker compose up` for the health check to pass

`ModuleNotFoundError: quanta` → Make sure the virtual environment is active: `source .venv/bin/activate` → Then run `pip install -e .` inside the repository root

`QuantaError: Neo4j not configured` → Expected when `NEO4J_URI` is not set in `.env` — the system automatically falls back to `NullGraph`

`turbovec compilation error` **during install** → The turbovec package requires the Rust toolchain:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

`DuckDBDocStore is not initialised` → You forgot to call `await docstore.init()` before the first operation

Results have `content=None` → The chunk was added to the index but not to the docstore — call `add_chunk()` alongside `idx.add()`

For the full configuration reference see USER_MANUAL.md.